

Zupurb Partner Roles — Functional & Non-Functional Spec

Version: 1.1 (code-grounded) · **Date:** 2026-06-22 **Companion to:** docs/PARTNER_ROLES_PLAN.md , docs/PARTNER_ROLES_SOW.md **Scope:** the **remaining (not-yet-built) roles** — Entertainer, Service Vendor, Brand/Sponsor, Influencer/Creator. (Venue Owner already shipped as `zupurb_owner` ; Admin/Staff is operational only and out of scope.)

Each role lists **functional features**, **non-functional requirements (NFRs)**, the **end-to-end flow**, the **consumer-side touchpoints today**, and a ⚠ **Build reality** callout — what already exists vs what must be built. The build-reality notes are grounded in the current codebase and are the most important part for planning.

Shared baseline (every partner role)

Category	Requirement
Security	A partner can read/write only its own entity + records — enforced by new <code>isPartner()</code> / <code>isPartnerOf(type, id)</code> rules + <code>partner</code> / <code>partnerType</code> custom claims (generalizing today's <code>businessOwner</code> + <code>isEstablishmentOwner</code>). Server-only fields (rating, verification, counts) stay non-client-writable.
Security	App Check enforced on callables; partner status granted only via admin-approved <code>claimRequests</code> — no self-promotion.
Reliability	State-changing callables are idempotent (reuse the existing <code>idempotencyKey</code> pattern from reservations/messages).
Performance	Lists paginate/limit (the consumer providers already use <code>limit 20</code>); analytics from precomputed aggregates, not heavy live queries.
Maintainability	Each web app is scaffolded by copying <code>zupurb_owner</code> ; shared auth/layout/Firebase pieces extracted into a common template at app #2.
Observability	Reuse <code>functions/src/lib/logging.ts</code> + notify utilities; emit structured logs + consumer notifications on partner actions.
Compliance	Responses/content pass the existing moderation pipeline; original consumer reviews are never editable by partners.

Entertainer → `zupurb_entertainer` **(web)**

A bookable performer (DJ, band, comedian, magician, dancer) who owns the profile consumers discover and book.

Functional features

- **Claim & verify** — register `accountType: 'entertainer'`, claim an `entertainers/{id}` doc via `claimRequests (entityType='entertainer')`; admin approval grants the partner claim.
- **Profile editor** — the fields the consumer renders: `name`, `role` (drives the "What they offer" chips), `tagline`, `city`, hero `imageUrl`, `isActive` toggle.
- **Replace synthesized content with real fields** — a stored `About`, explicit `services /offerings`, and `genres` (today these are faked from the role string).
- **Availability signal** — real "Available for bookings" flag (today hardcoded). Full calendar = later.
- **Booking & inquiry inbox** — accept / decline / quote / counter.
- **Messaging** — entertainer side of `entertainer_user` chats (infra exists).
- **Reviews** — view & respond (owner-style `ownerResponse`).
- **Dashboard stats** — bookings, unread inquiries, rating, active status.

Non-functional (role-specific): booking state machine prevents double-accept (idempotent `createBooking / respondToBooking`); profile edits reflect on the consumer rail promptly via existing snapshot streams; new `booking_request / booking_accepted` notification types.

Flow: sign up → claim profile → admin verifies → complete profile + `isActive=true` → set availability → consumer submits booking sheet (`createBooking`) → entertainer accepts/declines/quotes (`respondToBooking` , consumer notified) → chat to finalize → event → review → respond.

Consumer touchpoints today: "Featured artists & entertainers" rail + search rows (with "Book" buttons) in `search_screen.dart`; `entertainer_screen.dart` (`/entertainer/:id`); `booking_sheet.dart`; `entertainers_provider.dart` (streams `isActive==true` , limit 20); "Entertainers" tab in `messages_list_screen.dart` .

⚠ Build reality: The booking sheet's "Send request" **only fires a SnackBar — it writes nothing**; bookings need a new `bookings/{id}` collection + `createBooking / respondToBooking` callables. The detail screen **synthesizes** `About` from the tagline, **derives** services from the role, and **hardcodes** the availability banner. The "Message" button still shows **"coming soon."** `Reviews/ ReviewDoc` are **hardwired to establishments (estId)** and `entertainer.rating` has **no review pipeline** — the backend must extend reviews to target entertainers.

Service Vendor → `zupurb_vendor` **(web)**

A local event-services business — caterer, photographer, florist, event-rentals, bartending, bakery.

Functional features

- **Claim & verify** listing (`claimRequests entityType='vendor'`).
- **Profile editor** — `name`, `category` (6 seeded values), `tagline`, `imageUrl`, `city`, `isActive` .

- **Service packages & pricing** — net-new `vendors/{id}/packages` sub-collection (name, description, price/price-from, unit: per-event / per-head / per-hour). This is the MVP differentiator.
- **Inquiry inbox + quotes** — accept / decline / quote (reuses the `bookings` layer with a `vendorId`).
- **Messaging** — needs a new `vendor_user` conversation type.
- **Reviews** — view & respond.
- **Dashboard stats** — listing status, open inquiries, rating.

Non-functional (role-specific): `isPartnerOf('vendor', id)` write-gating; quoted pricing renders identically to the consumer (no stale prices); package builder usable by non-technical vendors; idempotent quote/accept.

Flow: sign in → claim listing → admin grants `partnerType: 'vendor'` + writes `partners/{uid}.managedIds=[vendorId]` → edit profile + `isActive` → create packages → receive inquiries → quote/accept/decline → chat → respond to reviews.

Consumer touchpoints today: `models/vendor.dart` , `vendors_provider.dart` (streams `isActive==true`), `vendor_card.dart` , shared `entity_detail_sheet.dart` ; 6 demo vendors seeded in `seedDemoData.ts` .

⚠ **Build reality: Vendors are not surfaced in ANY live consumer screen** — `search_screen.dart` renders only Places, Posts, and Entertainers, so `VendorCard` / `vendorsProvider` are **orphaned**. Launching this role requires building the **consumer-side vendor browsing + inquiry UI** (mirroring the entertainer rail + booking sheet) so the partner app has inbound demand. Packages are entirely net-new.

Brand / Sponsor → `zupurb_brand` (web)

A non-local product/lifestyle brand running point-redeemable offers and sponsored placements.

Functional features

- **Claim & verify** brand account (`claimRequests entityType='brand'`).
- **Brand profile editor** — `name` , `category` , `tagline` , `imageUrl` (logo), `city` , `isActive` .
- **Offer/deal composer** — reuse the deals engine `DealDoc` (title, description, `pointCost` , `originalValueCents` , cover image, `startsAt` / `expiresAt` , redemption caps, tier).
- **Offer lifecycle** — activate/deactivate/edit/delete (mirrors `zupurb_owner` /dashboard/deals).
- **Alcohol-compliance feedback** — surface the California ABC guard (`assertNotAlcoholDeal`) inline at compose time.
- **Sponsored placement requests** — request a featured Explore slot.
- **Redemption/performance stats** — `redemptionsCount` + `dealRedemptions` log.

Non-functional (role-specific): `isPartnerOf('brand', id)` gating; **California ABC alcohol rules are hard-blocked** at create + redeem (many brands are spirits); reuse engine safeguards (idempotency, atomic `remainingRedemptions` decrement, per-user/total caps); brand offers must populate the denormalized fields consumer deal queries depend on (`isActive` , `expiresAt` , `category`).

Flow: sign up → claim brand → admin grants `partnerType: 'brand'` → complete profile + `isActive` → compose offer (passes ABC guard) → publish + optionally request sponsored placement → appears in Home Deals tab → consumers redeem (points + QR) → brand monitors redemptions → adjust/deactivate.

Consumer touchpoints today: `models/brand.dart` + `brands_provider.dart`; `brand_card.dart` + `entity_detail_sheet.dart`; seeded brands in `seedDemoData.ts`; the **deals engine** (Home "Deals" tab, `deal_detail_screen.dart`, `redeemDeal` + QR redemption).

⚠ **Build reality:** Like vendors, `BrandCard` / `brandsProvider` are wired into **NO** screen (orphaned). Bigger engine gap: `DealDoc` is **establishment-scoped** (requires `estId/estName/estCity/geohash`), so brand offers need new `brandId/brandName` fields or must attach to a partner venue. **Sponsored placement is net-new** — there's no `isSponsored` / `priority` field and no Explore rail to render it; the consumer-side sponsored slot is a prerequisite.

★ Influencer / Creator → in-app (Flutter), not a web portal

A high-output reviewer/poster who enrolls in a Creator Program for a verified badge, analytics, and disclosure.

Functional features

- **Creator Program enrollment & status** — in-app apply flow; add a real role marker (extend `accountType` or a `creatorProgram` block on `UserDoc`: status `none/pending/approved/rejected`). Eligibility from existing counters (`reviewCount`, `verifiedReviewCount`, `followersCount`).
- **Verified Creator badge** — shown on profile + review author rows (reuse `badges` / `userBadges` or `isVerifiedCreator`).
- **Disclosure compliance** — persist the disclosure answers into `ReviewDoc.disclosureCategory` (enum `none|complimentary|media|employee|owner`) and render the label; mandatory for approved creators.
- **Post/review analytics** — review `helpfulScore` / upvotes, post `likeCount` / `commentCount`, follower growth (a `getCreatorStats` callable).
- **Real server-backed followers** — migrate follow from device-local to the `follows/` collection + `followersCount`.
- **Creator profile editor** — niche, social handles, real public `@handle`.
- **Featured/promoted opt-in** — opt content into editorial consideration (`ReviewDoc.isFeatured`, featured-creator feed item).

Non-functional (role-specific): creator status / `isVerifiedCreator` / follower counts are **server-only** (never client-writable); FTC-style disclosure **enforced**; follow migration must keep `followersCount` accurate under concurrent writes (atomic increments); tooling is **additive to existing consumer screens** (no separate app); preserve existing a11y (Semantics labels, 44px tap targets); emit analytics events.

Flow: surfaced in Home "Creators" tab → apply (writes `creatorProgram` status `pending`) → admin/eligibility approval (grants badge, migrates to server-backed follows) → set up niche/handle →

write reviews with **mandatory disclosure** persisted → real follow/unfollow grows audience → view analytics → opt content in for featuring → engage via chat inbox.

Consumer touchpoints today: Home "Creators" tab (`home_screen.dart` , `creators_provider.dart` — users sorted by `reviewCount`); `follow_provider.dart` ; `creator_disclosure_screen.dart` ; profile screens; `post.dart` `persona` lens; `posts_provider.dart` ; server `social/feed.ts` featured-creator injection.

⚠ **Build reality:** "Creator" today is just "any user sorted by `reviewCount` " — there's **no creator role, no `isVerifiedCreator` , no badge. Follow is device-local only** (`SharedPreferences`) — it never writes `follows/` or `followersCount` , and the profile shows a **hardcoded "8.4K"**; this must move server-side. The **disclosure screen is a pure mock** — `submitReview.ts` **hardcodes** `disclosureCategory: 'none'` . The public `@handle` is **faked by lowercasing** `displayName` . Featured-creator selection is gated on `isPlusSubscriber` , not a creator opt-in.

Merchant / Business Revenue Tier → Owner/Business Portal (web)

Merchant monetization layered on the existing Owner Portal — reusable for the Yovi Business Portal (client-requested).

Functional features

- **Subscription tiers** — free vs paid; billing + entitlement gating unlocks premium features.
- **Enhanced profile page** (paid) — richer media/menu, highlights, priority placement (Yelp-style).
- **Review & demographic insights** — per-category scores (Food/Service/Atmosphere/Value...) plus a breakdown **by demographic cohort** ("who rates you well/poorly"), reusing the existing per-cohort / People-Like-You engine.
- **Targeted promotion** — sponsored placement targeted by **area** (geo) + **demographic cohort**; campaign list + basic performance.

Non-functional (role-specific)

- *Privacy:* insights/cohort breakdowns shown **only above a minimum sample size**, aggregate-only — never individual reviewers.
- *Compliance:* **ad-policy/legal review** for demographic ad targeting; clear "sponsored" labeling.
- *Security:* subscription entitlements + insights gated to the merchant's own venue (`isPartnerOf`); billing verified server-side.
- *Reliability:* billing/entitlement state consistent (grace periods, downgrade handling).

Flow: merchant opens portal → picks a paid tier → billing provider processes → entitlements unlock → enrich enhanced profile → review the insights dashboard (category + cohort breakdowns above threshold) → compose an area+demographic-targeted promotion → it runs and reaches the target audience → track performance.

Consumer touchpoints: enhanced profiles + sponsored promotions surface in consumer discovery/Explore and venue detail; the data comes from the structured-review + cohort engine

consumers already feed.

⚠️ Build reality: Review responses, deals, and announcements **already exist** in the live Owner Portal (reuse). NEW to build: **billing/subscription + entitlement gating**, the **tiered profile**, the **insights dashboard** (cohort data exists via People-Like-You, but the merchant-facing analytics view is net-new), and **demographic ad targeting** (area targeting has geo already; demographic targeting generalizes the not-yet-built Brand sponsored placement). Needs a billing provider + an ad-targeting policy/legal check.

Summary

Role	Interface	Core theme	Biggest build gap to close first
Entertainer	<code>zupurb_entertainer</code> (web)	Availability + real bookings	Bookings don't persist (mock sheet) + reviews are establishment-only
Service Vendor	<code>zupurb_vendor</code> (web)	Packages + inquiries/quotes	Vendors aren't shown on any consumer screen yet
Brand / Sponsor	<code>zupurb_brand</code> (web)	Offers + sponsored placement	Deals are establishment-scoped; no sponsored slot exists
Influencer / Creator	In-app (Flutter)	Program + analytics + disclosure	Follow is device-local; disclosure is a mock; no creator role
Merchant Revenue Tier	Owner/Business Portal (web)	Subscription + insights + targeted ads	Needs billing + demographic-targeting policy/legal check